

GIRAI AI Assistant — Technical Overview

A high-level summary of how the on-site AI research assistant is built, for stakeholders and technical partners.

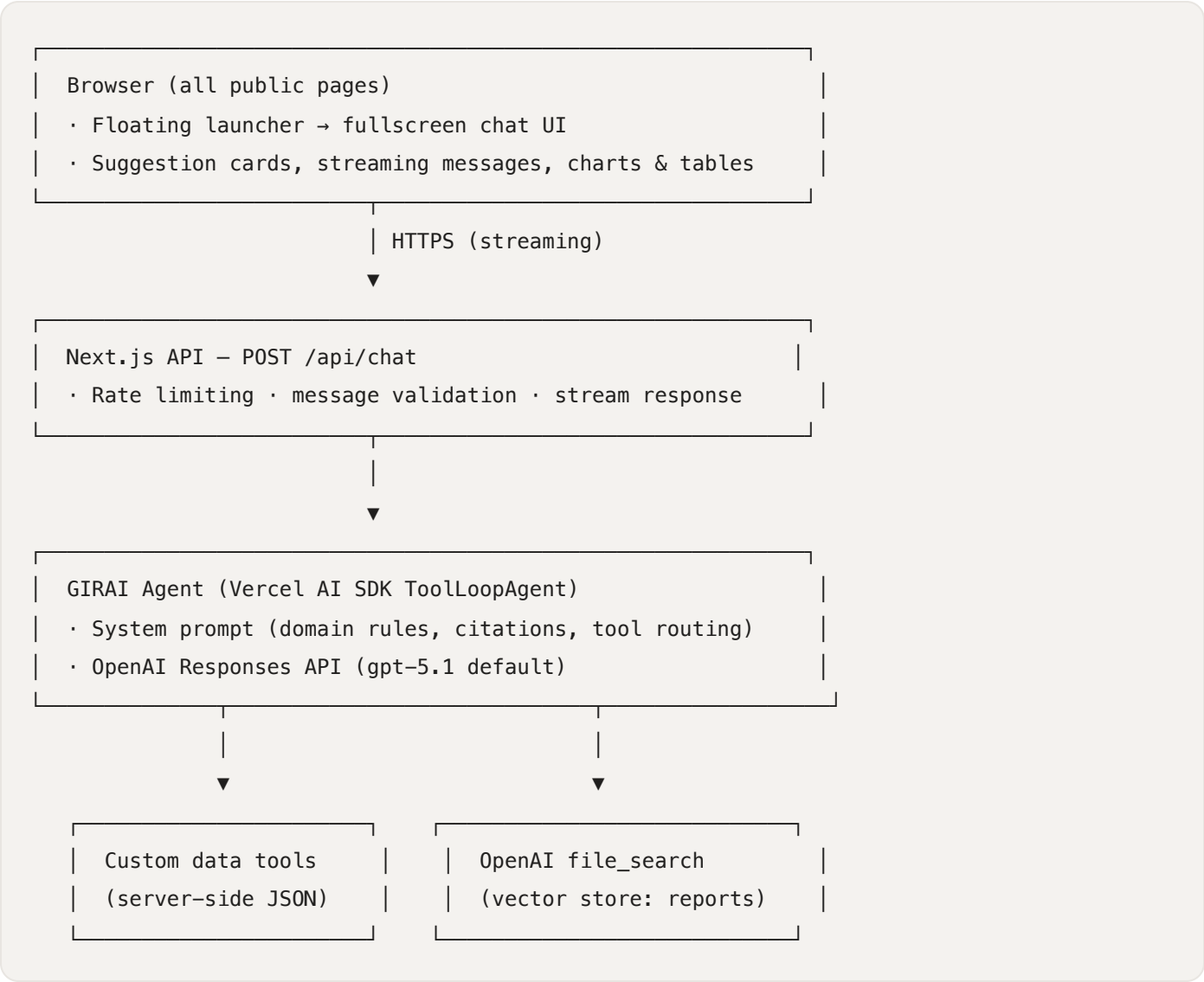
What it does

The **GIRAI Assistant** is a site-wide chat experience that helps visitors explore the Global Index on Responsible AI. Users can ask natural-language questions about:

- Country rankings and scores
- Indicators, dimensions, and pillars
- Evidence items (frameworks, initiatives, CSO work, etc.)
- Country and regional comparisons
- Changes between the 2024 and 2026 editions
- Narrative and methodology content from the published reports

Answers are grounded in **live GIRAI dataset JSON** (2026 scores and evidence) and **uploaded report content** (vector search over 2024/2026 report files). Responses include **clickable links** to relevant country, indicator, evidence, and region pages on the website.

Architecture at a glance



The assistant **does not guess numbers**. Factual queries trigger server tools that read the same committed dataset that powers the rest of the site. Qualitative or report-style questions use **semantic search** over uploaded PDF/report material in OpenAI’s vector store.

Core technologies

Layer	Technology	Role
Web framework	Next.js 16 (App Router)	Hosts pages and the <code>/api/chat</code> endpoint

Layer	Technology	Role
UI	React 19, Tailwind CSS 4, shadcn/ui	Layout, theming, accessible components
Chat UI	Vercel AI Elements	Conversation, messages, prompt input, reasoning display
AI runtime	Vercel AI SDK (ai , @ai-sdk/react , @ai-sdk/openai)	Agent loop, streaming, typed tool calls
Language model	OpenAI Responses API (gpt-5.1 by default; override with GIRAI_ASSISTANT_MODEL)	Reasoning, tool selection, natural-language answers
Report retrieval	OpenAI file search + vector store	Semantic search over 2024/2026 report uploads
Live data	GIRAI JSON (src/data/2026/generated/)	Authoritative scores, rankings, evidence
Charts	D3.js	Bar, line, and pie visualisations in chat
Motion	Motion (Framer Motion)	Launcher, suggestions, loading states

How answers are produced

1. System prompt

A curated instruction set defines the assistant’s role, GIRAI framework vocabulary (dimensions, pillars, indicators), data authority rules, citation format, and guardrails (no fabricated scores, no policy advice, 2026 vs 2024 edition rules).

2. Agent tools (live data)

Ten custom **server-side tools** wrap existing data accessors:

Tool	Typical use
lookup_country	Profile for one country
search_countries	Filter by region, subregion, income, score, or position vs the global average

Tool	Typical use
<code>get_leaderboard</code>	Top/bottom performers on GIRAI or an indicator, scopable to a region or subregion
<code>lookup_indicator</code>	Indicator definition and leaders
<code>search_evidence</code>	Find evidence by text, country, kind, region, or subregion
<code>compare_countries</code>	Side-by-side comparison (2–4 countries)
<code>get_averages</code>	Global, regional, subregional, or income-group averages and score-tier distribution
<code>get_edition_comparison</code>	2024 vs 2026 evidence-status changes
<code>get_region_summary</code>	Regional averages and rankings
<code>get_subregion_summary</code>	Subregional averages, ranks and member countries

Tool results include structured **data**, **source links** (e.g. `/countries/NGA`), and a hint for which **visualisation** to render (table, bar chart, comparison, etc.).

Geography arguments fail loudly. An unrecognised `region` or `subregion` returns an error naming the valid values — and, when the name is really a subregion, a hint to retry with the right parameter. Silently ignoring the filter used to return the whole index as though it were the requested slice ("which country leads LATAM" once answered with Norway at the top).

Subregions

`subregion` ships on every country record, but `scripts/build-data.ts` only pre-aggregates by region and income group. Subregional averages are therefore derived and memoized in `src/lib/girai/data.ts` (`getSubregionSummaries`).

Caribbean, Middle East and Northern America publish no subregional split; each resolves to a subregion of the same name, so every country belongs to exactly one subregion and the axis stays total. Several dataset labels differ from common usage — *South West Asia* is South Asia, *NorthWest Asia* is Central Asia plus the Caucasus, and the *South Africa* subregion means Southern Africa, not the country. `resolveSubregion` in `src/lib/ai/utils.ts` maps the common names onto the dataset ones.

2b. Evaluation

`pnpm eval:assistant` runs the cases in `scripts/eval/cases.ts` against the real agent — same instructions, tools and data as `/api/chat` — grades them deterministically, and writes a CSV under `results/` .

```
pnpm eval:assistant # default model
pnpm eval:assistant --model gpt-5-nano --repeat 3 # benchmark another model (lower --concurrent)
pnpm eval:assistant --only subregion # one case while iterating
```

Expected figures come from the committed dataset, not the published regional briefs; where the two disagree the dataset wins, and the case carries a `notes` line saying so. Numeric checks accept 2, 1 or 0 decimal places because the site renders one.

On the current 16-case "Rankings and scores" suite, the default model (`gpt-5.1`) scores **48/48** across three runs. `gpt-5-nano` (45/48) and `gpt-5.4-nano` (43/48) were benchmarked as cheaper alternatives; results are in `results/`.

3. File search (reports)

For methodology, narrative, and edition commentary, the agent uses OpenAI's `file_search` tool against a configured **vector store** containing the 2024 and 2026 report materials. This complements—but does not replace—live JSON for scores and rankings.

4. Streaming to the browser

The API streams the agent response in real time. The chat UI renders:

- Markdown text (with streaming)
- Tool progress and reasoning (when applicable)
- **Custom cards**: tables, bar/line/pie charts, comparison views, evidence lists
- **Sources** footer with links to site pages

User interface

Element	Description
Launcher	Fixed button (bottom-right) on all public pages; hidden on <code>/studio</code>
Fullscreen chat	Modal-style panel with header, scrollable conversation, and input dock
Suggestion cards	Starter questions grouped by theme (Compare, Rankings, Evidence, etc.)
Loading state	Animated shimmer and progress while the model/tools run
Insight cards	Shared visual shell for tables and charts inside assistant replies

Charts support switching views where appropriate (**Bars, Trend, Share**) for leaderboards, country lists, comparisons, and regional summaries.

Data & trust model

Data type	Source of truth
Scores, ranks, evidence counts	2026 GIRAI JSON (build-time dataset, same as the website)
2024 vs 2026 evidence changes	Edition comparison artifact (not cross-edition score comparison)
Report prose & methodology	Vector store (uploaded reports)
Page links	Generated from ISO3 codes, indicator slugs, evidence IDs, region slugs

The assistant is instructed to **cite sources** and link to real pages rather than invent URLs or statistics.

Cost of running the assistant

The assistant runs on OpenAI's pay-as-you-go API: **you are billed per use, not a flat subscription**. Cost is driven by how much text goes in and out of the model, measured in *tokens* (roughly $\frac{3}{4}$ of a word). Every question sends in the assistant's instructions, the relevant slice of GIRAI data it looked up, and the conversation so far (the "input"), and the model sends back its answer (the "output").

There is **no per-page or idle cost** — the model only runs when a visitor actually asks a question. Loading a country page, browsing rankings, or leaving the chat open costs nothing.

The model

We currently run **GPT-5.1**, OpenAI's newer, more capable model. It scored **100% on our accuracy test** (see the sibling *Accuracy Improvements & Benchmark* report). The model is set in one place (`GIRAI_ASSISTANT_MODEL`) and can be swapped without code changes. For context, here's how it compares on price with the older GPT-4.1 we ran previously:

	Price per 1 million input tokens	Price per 1 million output tokens
GPT-5.1 (current)	\$1.25	\$10.00
GPT-4.1 (previous)	\$2.00	\$8.00

GPT-5.1 is **cheaper to send data into** (input) but **a little pricier for the answers it writes** (output). Because a typical GIRAI question sends in far more than it writes back (all that data lookup), **GPT-5.1 works out slightly cheaper per question** — while also being the smarter, more accurate model. Prices are OpenAI's public rates as of this writing and can change; check [OpenAI's pricing page](#) for the latest.

A note on "caching": the assistant's fixed instructions are the same on every request. OpenAI charges a large discount (roughly 75–90% off) for that repeated portion, so real-world costs tend to land at the low end of the estimates below.

What a question actually costs

A single question isn't one model call — the assistant may look data up, read the result, and then write its answer (up to a dozen internal steps). Bundling that together, a typical question uses very roughly **8,000–15,000 input tokens and 400–900 output tokens**. That works out to:

Question type	GPT-5.1 (current)	GPT-4.1 (previous)
Simple (one lookup, e.g. "What's Kenya's score?")	~\$0.01	~\$0.01
Typical (a comparison or ranked list)	~\$0.02	~\$0.03
Heavy (multi-country + report search)	~\$0.04	~\$0.06

Rule of thumb: a few cents per question. Even a busy day of a few hundred questions is single-digit dollars.

Monthly cost at different traffic levels

Assuming an average "typical" question:

Questions per month	GPT-5.1 (current)	GPT-4.1 (previous)
1,000	~\$20	~\$30
5,000	~\$110	~\$150
10,000	~\$220	~\$300

These are estimates to plan around, not a bill — actual spend depends on question complexity and length. OpenAI provides a live usage dashboard, and you can set a **hard monthly spending cap** so costs can never run away.

Built-in cost controls

The assistant already limits spend in several ways:

- **Rate limiting** — max 30 questions per minute from any single visitor, blocking runaway or abusive use.
- **Conversation trimming** — only the last 40 messages are kept, so long chats don't keep re-sending an ever-growing history.
- **Step cap** — the model can take at most 12 internal steps per question.
- **Data lookups are free** — the tools read our own JSON files; only the model calls cost money.

One-time build cost (country narratives)

Separately, the written summaries on country pages are generated **once, during a build**, using the cheaper `gpt-4o-mini` model (about one small call per country). A full regeneration of all 135 countries costs **under \$0.10 total**, and it never runs on the live site. See [country-page-narratives-guide.md](#).

Security & operations

- **API keys** (`OPENAI_API_KEY` , `OPENAI_VECTOR_STORE_ID`) live in server environment variables only—never exposed to the browser.
- `/api/chat` applies basic **rate limiting** (per IP) and caps conversation history length.
- Tools read **static JSON only**; there is no arbitrary database or file access from the model.
- The default model (`gpt-5.1`) and vector store ID can be overridden via `GIRAI_ASSISTANT_MODEL` and `OPENAI_VECTOR_STORE_ID` .

See `.env.local.example` for required configuration.

Key code locations

Area	Path
Agent & tools	<code>src/lib/ai/</code>
Chat API	<code>src/app/api/chat/route.ts</code>
Chat UI	<code>src/components/ai-assistant/</code>
Charts & tables	<code>src/components/ai-assistant/viz/</code>
Site integration	<code>src/app/layout.tsx</code> (<code>AiAssistantProvider</code>)

Dependencies (AI-specific)

ai	– Vercel AI SDK (agent, streaming)
@ai-sdk/openai	– OpenAI provider (Responses API, file search)
@ai-sdk/react	– useChat hook for the client
ai-elements	– Pre-built chat UI components (installed into the repo)
d3	– Chart rendering
motion	– UI animation
streamdown	– Markdown rendering in messages

Possible future extensions

- Code interpreter for ad-hoc analysis of uploaded spreadsheets
 - Conversation persistence or export
 - Analytics on common questions
 - CMS-driven suggestion copy
 - Additional chart types or map snippets in replies
-